第9章 作业 9.3 9.31 9.33

第10章作业预告：

10.1 10.6 10.8 10.25 10.42

数据结构课程的内容

- 数据结构
  - 逻辑结构
    - 线性结构（线性表、栈　、队、串、数组）
    - 非线性结构
      - 树结构
      - 图结构
  - 物理（存储）结构
    - 顺序结构
    - 链式结构
    - 索引结构
    - 散列结构
  - 数据运算
    - 插入运算
    - 删除运算
    - 修改运算
    - 查找运算
    - 排序运算

第10章 内部排序

10.1 概述
10.2 插入排序
10.3 交换排序
10.4 选择排序
10.5 归并排序
10.6 基数排序

10.1 概述

1. 什么是排序？

将一组杂乱无章的数据按一定的规律顺次排列起来。

存放在数据表中

按关键字排序

2. 排序的目的是什么？ ——便于查找！

3. 排序算法的好坏如何衡量？

- **时间效率——排序速度（即排序所花费的全部比较次数）**
- **空间效率——占内存辅助空间的大小**
- **稳定性——若两个记录A和B的关键字值相等，但排序后A、B的先后次序保持不变，则称这种排序算法是稳定的。**

4. 什么叫内部排序？什么叫外部排序？

——若待排序记录都在内存中，称为内部排序；

——若待排序记录一部分在内存，一部分在外存，则称为外部排序。

注：外部排序时，要将数据分批调入内存来排序，中间结果还要及时放入外存，显然外部排序要复杂得多。

5. 待排序记录在内存中怎样存储和处理？

① 顺序排序——排序时直接移动记录；

② 链表排序——排序时只移动指针；

③ 地址排序——排序时先移动地址，最后再移动记录。

注：地址排序中可以增设一维数组来专门存放记录的地址。

6. 顺序存储（顺序表）的抽象数据类型如何表示？

注：大多数排序算法都是针对顺序表结构的(便于直接移动元素)

```
# define MAXSIZE 20        //设记录数不超过20个（本章都如此假设）
typedef  int  KeyType ;     //设关键字为整型量（int型)
```

```
Typedef  struct {                //定义每个记录（数据元素）的结构
   KeyType      key ;            //关键字
   InfoType     otherinfo;       //其它数据项
}RecordType，  node ;            //例如node.key表示其中一个分量
```

```
Typedef  struct {                          //定义顺序表L的结构
 RecordType  r [ MAXSIZE +1 ];   //存储顺序表的向量
                                 //r[0]一般作哨兵或缓冲区
 int length ;        //顺序表的长度
}SqList ,L;          //例如L.r或L.length表示其中一个分量
```

若r数组是表L中的一个分量且为node类型，
则r中某元素的key分量表示为：r[i].key

7. 内部排序的算法有哪些？

——按排序的规则不同，可分为5类：

- 插入排序
- 交换排序（重点是快速排序）
- 选择排序
- 归并排序
- 基数排序

——按排序算法的时间复杂度不同，可分为3类：

- 简单的排序算法：时间效率低，$O(n^2)$
- 先进的排序算法：时间效率高，$O(n\log_2 n)$
- 基数排序算法：时间效率高，$O(d\times n)$

d＝关键字的位数(长度)

10.2 插入排序

插入排序的基本思想是：每步将一个待排序的对象，按其关键码大小，插入到前面已经排好序的一组对象的适当位置上，直到对象全部插入为止。

简言之，边插入边排序，保证子序列中随时都是排好序的。

插入排序有多种具体实现算法：

1) 直接插入排序
2) 折半插入排序
3）2-路插入排序
4) 表插入排序

} 小改进

5) 希尔排序 → 大改进

1) 直接插入排序

最简单的排序法！

新元素插入到哪里？ 在已形成的有序表中线性查找，并在适当位置插入，把原来位置上的元素向后顺移。

例1： 关键字序列T=（13，6，3，31，9，27，5，11），请写出直接插入排序的中间过程序列。

【13】, 6, 3, 31, 9, 27, 5, 11

【6, 13】, 3, 31, 9, 27, 5, 11

【3, 6, 13】, 31, 9, 27, 5, 11

【3, 6, 13，31】, 9, 27, 5, 11

【3, 6, 9, 13，31】, 27, 5, 11

【3, 6, 9, 13，27, 31】, 5, 11

【3, 5, 6, 9, 13，27, 31】, 11

【3, 5, 6, 9, 11，13，27, 31】

直接插入排序算法的实现：　　(参见教材P265)

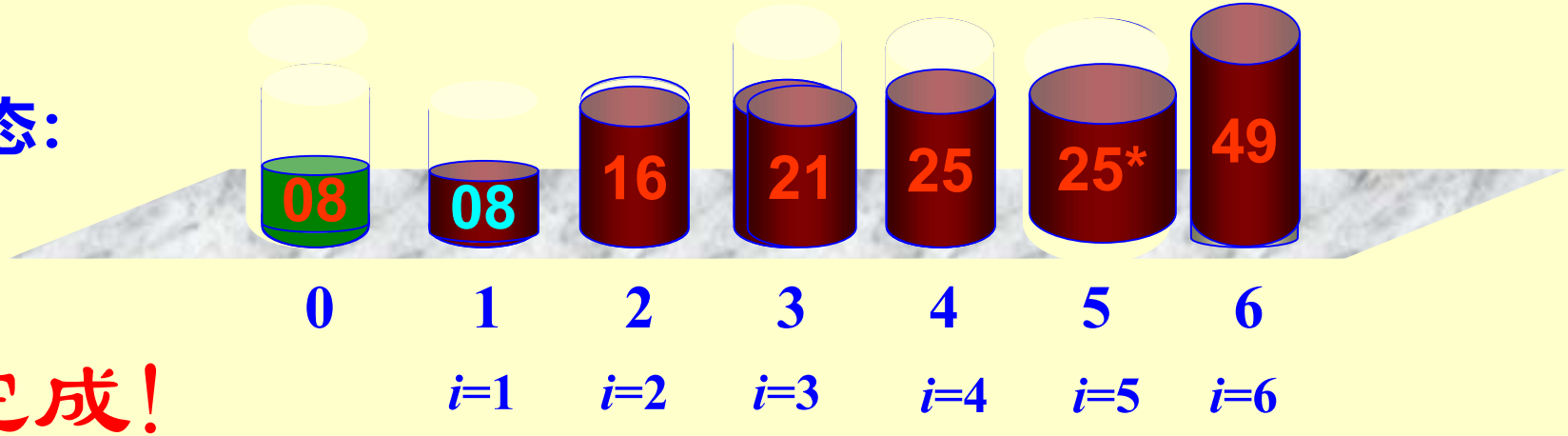
```
void InsertSort ( SqList &L ) { //对顺序表L作直接插入排序
   for ( i = 2;  i <=L.length; ++ i ) //直接在原始无序表L中排序
 if (L.r[i].key < L.r[i-1].key) //若L.r[i]较小则插入有序子表内
{ L.r[0]= L.r[i];            //先将待插入的元素放入“哨兵”位置
  L.r[i]= L.r[i-1];         //子表元素开始后移
  for ( j=i-2; L.r[0].key < L.r[j].key; --j ) L.r[j+1]= L.r[j];
     //只要子表元素比哨兵大就不断后移
L.r[j+1]= L.r[0];      //直到子表元素小于哨兵，将哨兵值送入
                       //当前要插入的位置（包括插入到表首）
} //if
}// InsertSort
```

例1：关键字序列T=（21，25，49，25*，16，08），请写出直接插入排序的具体实现过程。

*表示后一个25

解：假设该序列已存入一维数组r[7]中，将r[0]作为哨兵（Temp）。则程序执行过程为：

初态：



完成！

时间效率：因为在最坏情况下，所有元素的比较次数总和为 $(0+1+\ldots+n-1)\rightarrow O(n^2)$。其他情况下也要考虑移动元素的次数。故时间复杂度为$O(n^2)$

空间效率：仅占用1个缓冲单元——O（1）

算法的稳定性：因为25*排序后仍然在25的后面——稳定

2） 折半插入排序

一个容易想到的改进方法：既然子表有序且为顺序存储结构，则插入时采用折半查找定可加速。

对应程序见教材**P267**（仅用于顺序表）

优点：**比较次数大大减少，全部元素比较次数仅为O($n\log_2 n$)。**

时间效率：**虽然比较次数大大减少，可惜移动次数并未减少，所以排序效率仍为O(n^2) 。**

空间效率：仍为 **O（1）**

稳 定 性： 稳定

思考：折半插入排序还可以改进吗？
能否减少移动次数？

3） 2-路插入排序

这是对折半插入排序的一种改进，其目的是减少排序过程中的移动次数。

代价：需要增加n个记录的辅助空间。

思路：增开辅助数组d，大小与r相同。

首先将r[1]赋值给d[1]，然后以d[1]内容为中值，将r[i]元素逐个插入到d[1]值之前或之后的有序序列中。

计算机处理时，可把d设为循环向量，再设头尾两个指针。

例：待排序列T=(49，38，65，97, 76, 13, 27, 49*，55，04）

排序中途: d= (49，65，76, 97), (13, 27, 38 ）

↑final ↑first

（参见教材P267—268例）

讨论：若记录是链表结构，用直接插入排序行否？

答：行，而且无需移动元素，时间效率更高！

但请注意：单链表结构无法实现“折半查找”

4）表插入排序

基本思想：**在顺序存储结构中，给每个记录增开一个指针分量，在排序过程中将指针内容逐个修改为已经整理（排序）过的后继记录地址。**

优点：**在排序过程中不移动元素，只修改指针。**

此方法具有链表排序和地址排序的特点

例： 关键字序列 T=(21，25，49，25*，16，08），
请写出表插入排序的具体实现过程。

*表示后一个25

解：假设该序列（结构类型)已存入一维数组r[7]中，将r[0]作为表头结点。则算法执行过程为：

	i	关键字 r[i].key	指针 r[i].link
初态	0	MaxNum	6
i=1	1	21	2
i=2	2	25	4
i=3	3	49	0
i=4	4	25*	3
i=5	5	16	1
i=6	6	08	5

表头结点

指向第1个元素

表插入排序的算法

```
int LinkInsertSort (Linklist &L) {
    L.r[0].Key = MaxNum;
    L.r[0]. Link = 1;
    L.r[1].Link = 0;                    //形成循环链表
for ( int i = 2;  i <= L.length; i++ ) {
        int  current = L.r[0]. Link;      //current=当前记录指针
        int pre = 0;                      //pre=当前记录current的前驱指针
    while ( L.r[current]. Key <= L.r[i]. Key) {
      pre = current;              //current指针准备后移, pre跟上;
     current = L.r[current]. Link; }  //找插入位置(即p=p->link)

    L.r[i]. Link = current; //新记录r[i]找到合适序位开始插入
    L.r[pre]. Link = i;    //在pre与current之间链入
        } //for
    } //LinkInsertSort
```

表插入排序算法分析：

① 无需移动记录，只需修改2n次指针值。但由于比较次数没有减少，故时间效率仍为$O(n^2)$ 。

② 空间效率肯定低，因为增开了指针分量（但在运算过程中没有用到更多的辅助单元）。

③ 稳定性：25和25*排序前后次序未变，稳定。

注：此算法得到的只是一个有序链表，查找记录时只能满足顺序查找方式。

改进：可以根据表中指针线索，很快对所有记录重排，形成真正的有序表（顺序存储方式），从而能满足折半查找方式。具体实现见教材P269。

5）希尔（shell）排序　又称缩小增量排序

基本思想：先将整个待排记录序列分割成若干子序列，分别进行直接插入排序，待整个序列中的记录“基本有序”时，再对全体记录进行一次直接插入排序。

技巧：子序列的构成不是简单地“逐段分割”，而是将相隔某个增量dk的记录组成一个子序列，让增量dk逐趟缩短（例如依次取5, 3, 1），直到dk＝1为止。

优点：让关键字值小的元素能很快前移，且序列若基本有序时，再用直接插入排序处理，时间效率会高很多。

例：关键字序列 T=(49，38，65，97, 76, 13, 27, 49*，55，04），请写出希尔排序的具体实现过程。

r[i]	0	1	2	3	4	5	6	7	8	9	10
初态:		49	38	65	97	76	13	27	49*	55	04
第1趟 (dk=5)		13	27	49*	55	04	49	38	65	97	76
第2趟 (dk=3)		13	04	49*	38	27	49	55	65	97	76
第3趟 (dk=1)		04	13	27	38	49*	49	55	65	76	97

算法分析：开始时*dk* 的值较大，子序列中的对象较少，排序速度较快；随着排序进展，*dk* 值逐渐变小，子序列中对象个数逐渐变多，由于前面工作的基础，大多数对象已基本有序，所以排序速度仍然很快。

时间效率： $O(n^{1.25})$ ~ $O(1.6n^{1.25})$ ——由经验公式得到

空间效率：O（1）——因为仅占用1个缓冲单元

算法的稳定性：不稳定——因为49*排序后却到了49的前面

课堂练习：

练1. 欲将序列（Q, H, C, Y, P, A, M, S, R, D, F, X）中的关键码按字母升序重排，则初始步长为4的希尔排序一趟的结果是？

答：**原始序列：** Q, H, C, Y, P, A, M, S, R, D, F, X

shell一趟后： P, A, C, S, Q, D, F, X, R, H, M, Y

练2. 以关键字序列（256，301，751，129，937，863，742，694，076，438）为例，写出执行希尔排序（取dk=5, 3, 1）算法的各趟排序结束时，关键字序列的状态。

原始序列： 256，301，751，129，937，863，742，694，076，438

希尔排序

第1趟 256，301，694，076，438，863，742，751，129，937
dk=5

第2趟 076，301，129，256，438，694，742，751，863，937
dk=3

第3趟 076，129，256，301，438，694，742，751，863，937
dk=1

(取dk=5, 3, 1)

希尔排序算法（主程序）

参见教材P272

```
void   ShellSort(SqList &L，int dlta[ ]，int t)
{            //按增量序列dlta[0…t-1]对顺序表L作Shell排序
  for(k=0；k<t；++k)
    {
      ShellInsert(L，dlta[k]);   //增量为dlta[k]的一趟插入排序
    }
} // ShellSort
```

dk值依次装在dlta[t]中

希尔排序算法（其中某一趟的排序操作）

参见教材P272

```
void  ShellInsert(SqList &L，int dk)
{  //对顺序表L进行一趟增量为dk的Shell排序，dk为步长因子
   for(i=dk+1；i<=L.length；  ++ i)
      if(r[i].key < r[i-dk].key)    //开始将r[i] 插入有序增量子表
       { r[0]=r[i]；                //暂存在r[0]，此处r[0]仍是哨兵
         for(j=i-dk; j>0&&(r[0].key<r[j].key); j-=dk) r[j+dk]=r[j]；
                                    //关键字较大的记录在子表中不断后移
       r[j+dk]=r[0]；               //在本趟结束时将r[i]插入到正确位置
      }//if
} //ShellInsert
```

理解难点：整理动作是二合一的， r[0]仍是每个dk子集的哨兵，用于子集的彻底排序！

对前页程序中间for循环的理解：

空间效率与算法设计有关

```
for(i=dk+1；i<=L.length；  ++ i)
    if(r[i].key < r[i-dk].key)
     { r[0]=r[i]；
for(j=i-dk; j>0&&(r[0].key<r[j].key); j-=dk)  r[j+dk]=r[j]；
r[j+dk]=r[0]；  }
```

大者后移

如果没有此句，就只能实现相临的两个数之间的比较，而不能两两都比较到，所以有可能达不到排序的目的。

例如当dk=5时，第一趟比较之前的原始子集是：

5	7	4
i=1	i+dk=6	i+2dk=11

如果不用for循环，比较的结果是 5，4，7

只有执行for循环后，比较结果才会是 4，5，7

10.3 交换排序

交换排序的基本思想是：

两两比较待排序记录的关键码，如果发生逆序（即排列顺序与排序后的次序正好相反），则交换之，直到所有记录都排好序为止。

交换排序的主要算法有：

1) 冒泡排序

2) 快速排序

1) 冒泡排序

基本思路： 每趟不断将记录两两比较，并按“前小后大”（或“前大后小”）规则交换。

优点： **每趟结束时，不仅能挤出一个最大值到最后面位置，还能同时部分理顺其他元素；一旦下趟没有交换发生，还可以提前结束排序。**

前提： 顺序存储结构

例： 关键字序列 T=(21，25，49，25*，16，08），请写出冒泡排序的具体实现过程。

初态：	21，	25，	49，	25*，	16，	08
第1趟	21，	25，	25*，	16，	08，	49
第2趟	21，	25，	16，	08，	25*，	49
第3趟	21，	16，	08，	25，	25*，	49
第4趟	16，	08，	21，	25，	25*，	49
第5趟	08，	16，	21，	25，	25*，	49

冒泡排序的算法分析

•**最好情况：** 初始排列已经有序，只执行一趟起泡，做 n-1 次关键码比较，不移动对象。

•**最坏情形：** 初始排列逆序，算法要执行n-1趟起泡，第i趟 ($1 \le i < n$) 做了$n-i$ 次关键码比较，执行了$n-i$ 次对象交换。此时的比较总次数KCN和记录移动次数RMN为：

$$KCN = \sum_{i=1}^{n-1}(n-i) = \frac{1}{2}n(n-1)$$

$$RMN = 3\sum_{i=1}^{n-1}(n-i) = \frac{3}{2}n(n-1)$$

因此：

时间效率：O（n^2）—因为要考虑最坏情况

空间效率：O（1） —只在交换时用到一个缓冲单元

稳 定 性： 稳定 —25和25*在排序前后的次序未改变

冒泡排序的优点：每一趟整理元素时，不仅可以完全确定一个元素的位置（挤出一个泡到表尾），还可以对前面的元素作一些整理，所以比一般的排序要快。

有没有比冒泡排序更快的算法?

有!

快速排序法——全球公认!

因为它每趟都能准确定位不止1个元素!